



UNIVERZITET U SARAJEVU
ELEKTROTEHNIČKI FAKULTET
ODSJEK ZA RAČUNARSTVO I INFORMATIKU

**QR ALGORITAM ZA NALAŽENJE
SVOJSTVENIH VRIJEDNOSTI MATRICA UZ
PRETHODNO SVOĐENJE MATRICE NA
HESSENBERGOVU NORMALNU FORMU**

Student: *Hana Helac*

Mentor: *red. prof. dr. Željko Jurić, dipl. ing. el.*

Numerički algoritmi

Sarajevo, februar 2026

Sažetak

QR algoritam je standardna numerička metoda za određivanje svojstvenih vrijednosti kvadratnih matrica. Efikasnost postupka se postiže prethodnim svođenjem matrice na gornju Hessenbergovu normalnu formu pomoću ortogonalnih transformacija, pri čemu se spektar matrice ne mijenja, a svaka naredna QR iteracija postaje znatno jeftinija. Opisan je princip QR iteracije i njena veza sa Schurovom formom, uz naglasak na praktične mehanizme ubrzanja konvergencije kao što su pomaci (shiftovi) i deflacija. Razmatraju se osnovni implementacijski aspekti, uključujući kriterije zaustavljanja, izbor tolerancije i provjeru tačnosti rezultata poređenjem sa referentnim rješenjima.

Ključne riječi: QR algoritam, Hessenbergova forma, svojstvene vrijednosti, Schurova forma, Householder, shift, deflacija.

Abstract

The QR algorithm is a standard numerical method for determining the eigenvalues of square matrices. The efficiency of the procedure is achieved by first reducing the matrix to upper Hessenberg form using orthogonal transformations; this preserves the spectrum while making each subsequent QR iteration significantly cheaper. The basic idea of the QR iteration and its connection to the Schur form are described, with emphasis on practical convergence-acceleration mechanisms such as shifts and deflation. Key implementation aspects are also considered, including stopping criteria, tolerance selection, and validation of the results by comparison with reference solutions.

Keywords: QR algorithm, Hessenberg form, eigenvalues, Schur form, Householder, shift, deflation.

Sadržaj

Sažetak

Abstract

Poglavlje 1 – Uvod	1
1.1 Teorijski uvod.....	1
1.2 Svojstvene vrijednosti.....	1
1.3 QR faktorizacija.....	2
1.4 Hessenbergova forma.....	4
1.5 QR algoritam.....	6
1.6 Vremenska kompleksnost QR algoritma.....	9
 Poglavlje 2 – Implementacija algoritma.....	 11
2.1 Osnovna verzija QR algoritma.....	11
Testni primjer 1 – Simetrična matrica.....	13
Testni primjer 2 – Nesimetrična matrica.....	15
Testni primjer 3 – Pascalova matrica.....	16
Testni primjer 4 – Hilbertova matrica.....	17
 Poglavlje 3 – Primjena algoritma u praksi.....	 19
 Poglavlje 4 – Zaključak i diskusija.....	 21
 Reference.....	 2

Poglavlje 1

Uvod

1.1 Teorijski uvod

Računanje svojstvenih vrijednosti matrica je jedan od najvažnijih problema numeričke linearne algebre i pojavljuje se u velikom broju primjena u inženjerstvu i računarstvu. Stabilnost linearnih dinamičkih sistema, modalna analiza mehaničkih i električnih struktura, obrada signala, metode redukcije dimenzije, te brojni problemi diskretizacije diferencijalnih jednačina, u osnovi se svode na analizu spektra matrice. Zbog toga je važno imati metode koje su istovremeno dovoljno tačne, numerički stabilne i efikasne za matrice većih dimenzija.

Iako se svojstvene vrijednosti mogu definisati preko karakterističnog polinoma, direktno računanje na taj način u praksi nije pogodno, jer je postupak računski skup i osjetljiv na greške zaokruživanja, posebno kod matrica većeg reda. Samim tim, ovaj pristup je dosta numerički nestabilan i sasvim nepouzdan. Savremeni pristup zasniva se na faktorizacijama i transformacijama koje čuvaju spektar, a među njima je i QR algoritam kojim se bavi ovaj rad.

Prije detaljnije diskusije, potrebno je definirati osnovne pojmove kojima ćemo se koristiti kroz ovaj rad, te razjasniti osnovne koncepte linearne algebre potrebne za shvatanje datog algoritma.

1.2 Svojstvene vrijednosti

Prije formalnog izlaganja QR algoritma potrebno je precizno definisati pojmove svojstvenih vrijednosti i svojstvenih vektora, jer oni predstavljaju osnovni objekat računanja u ovom radu. Uvođenjem ovih definicija, jasno naglašavamo šta se

podrazumijeva pod spektrom matrice i na koji način se svojstvene vrijednosti povezuju sa djelovanjem linearne transformacije na vektorski prostor.

Neka je $A \in \mathbb{C}^{m \times m}$ kvadratna matrica. Nenulti vektor $x \in \mathbb{C}^m$ je **svojstveni vektor matrice A**, a $\lambda \in \mathbb{C}$ je odgovarajuća **svojstvena vrijednost**, ako vrijedi

$$Ax = \lambda x.^1$$

Osnovna ideja je da djelovanje matrice A na neki potprostor $S \subset \mathbb{C}^m$ u određenim situacijama može biti ekvivalentno množenju skalarom. Kada je to slučaj, potprostor S naziva se svojstveni potprostor, a svaki nenulti vektor $x \in S$ predstavlja svojstveni vektor. Svojstvene vrijednosti λ su upravo oni skalarni faktori za koje postoji nenulti vektor x takav da vrijedi $Ax = \lambda x$, kao što je već navedeno. Ekvivalentno, možemo ih definisati kao rješenja jednačine $\det(A - \lambda I) = 0$. Skup svih svojstvenih vrijednosti matrice A naziva se **spektar matrice A**.

Svojstvene vrijednosti i svojstveni vektori su važni iz više razloga. S jedne strane, u algoritamskom smislu omogućavaju pojednostavljivanje određenih problema, jer složene povezane sisteme često svode na analizu jednostavnijih komponenti. S druge strane, u fizičkom kontekstu spektralna analiza daje direktan uvid u ponašanje sistema opisanih linearnim modelima, posebno u pogledu rezonancije i stabilnosti. Upravo iz tih razloga bilo je važno razviti efikasne i numerički stabilne metode za njihovo računanje, poput QR algoritma uz prethodnu redukciju na Hessenbergovu formu, koje omogućavaju pouzdano računanje spektra i za matrice većih dimenzija.

1.3 QR faktorizacija

Budući da je QR faktorizacija osnovni gradivni blok QR algoritma, najprije je potrebno uvesti njenu definiciju i objasniti svojstva faktora koji se u njoj pojavljuju.

Za matricu A kaže se da ima **QR faktorizaciju** ako se može zapisati u obliku

¹ L. N. Trefethen and D. Bau III, *Numerical Linear Algebra*. Philadelphia, PA, USA: SIAM, 1997, str. 181.

$$A = QR$$

gdje je Q **ortogonalna matrica**, a R **gornja trougaona matrica**. Ortogonalnost matrice Q znači da vrijedi $Q^T Q = I$, pri čemu je I jedinična matrica, što direktno implicira da je transponovana matrica Q jednaka njenoj inverznoj matrici. Ovo svojstvo ukazuje na to da su kolone matrice Q međusobno ortogonalne, odnosno da je skalarni produkt bilo koje dvije različite kolone jednak nuli. Pored toga, svaka kolona ima normu jednaku 1, pa su kolone ortonormirane, kao i redovi.

QR faktorizacija nije ograničena samo na kvadratne matrice. Ona se može definisati i za pravougaone matrice koje imaju više redova nego kolona, tj. za $A \in \mathbb{R}^{m \times n}$ sa $m > n$. U tzv. punoj QR faktorizaciji tada se dobija ortogonalna matrica Q dimenzija $m \times m$, dok je R dimenzija $m \times n$ i ima gornjetrougaonu strukturu. Pošto je broj redova veći od broja kolona, dio matrice R koji “prelazi” preko dimenzije n ne može sadržavati nenulte elemente, te su zbog toga posljednjih $m-n$ redova matrice R jednaki nuli.

Zbog praktičnih razloga, posebno kod većih dimenzija, često se koristi i reducirana QR faktorizacija. U toj varijanti se ne formira kompletna matrica Q , nego se uzima samo dio koji je potreban, te je u ovoj varijanti matrica Q dimenzija $m \times n$ i ima ortonormirane kolone, dok je R kvadratna matrica dimenzija $n \times n$. Ovakav zapis je ekvivalentan punoj faktorizaciji za slučaj $m > n$, ali je ekonomičniji za računanje i skladištenje.²

QR faktorizacija se u praksi može dobiti na više načina, pri čemu su na kursu obrađeni najznačajniji postupci, poput Gram–Schmidtove ortogonalizacije i Householderovog algoritma. U okviru QR algoritma ova faktorizacija se koristi kao osnovni korak iteracije, dok izbor konkretne procedure za njeno računanje utiče prvenstveno na numeričku stabilnost i efikasnost implementacije. S obzirom da je fokus rada na QR algoritmu za svojstvene vrijednosti uz prethodnu redukciju na Hessenbergovu formu, detaljna razrada postupaka za konstrukciju QR faktorizacije neće se posebno

² Ž. Jurić, *Numerički algoritmi*. Sarajevo: Univerzitet u Sarajevu, Elektrotehnički fakultet, 2018, str. 121

razmatrati, već će se osloniti na standardne, dobro poznate metode iz literature i nastave.

1.4 Hessenbergova forma

Prije nego što pređemo na definisanje Hessenbergove forme i opisivanjem procesa redukcije na nju, bitno je da uvedemo i objasnimo šta je to balansiranje, čemu služi i zašto je relevantno.

Balansiranje se u numeričkoj linearnoj algebri često uvodi kao jedan od pripremnih postupaka prije primjene metoda za računanje svojstvenih vrijednosti opštih odnosno nesimetričnih matrica. Motiv za njegovo korištenje je činjenica da su greške usljed zaokruživanja u numeričkim postupcima u pravilu povezane s veličinom matrice, tj. njenom normom, što je norma veća, to je veći potencijalni uticaj zaokruživanja na konačni rezultat. Osnovna ideja balansiranja je da se pomoću transformacije ekvivalencija po sličnosti preoblikuje matrica tako da odgovarajući redovi i kolone imaju uporedive norme. Time se često smanjuje ukupna norma matrice, dok se svojstvene vrijednosti pri tome ne mijenjaju, pa se kasniji proračun spektra može učiniti pouzdanijim, naročito kod “loše izbalansiranih” matrica. Simetrične matrice u tom smislu ne zahtijevaju dodatno balansiranje, jer se smatraju već balansiranim.

Praktična prednost ovog postupka je i u tome što je računski relativno jeftin: tipično zahtijeva $O(n^2)$ operacija, pa njegov doprinos ukupnom vremenu obrade ostaje mali u poređenju s glavnim numeričkim koracima koji slijede. Nakon balansiranja, prirodan sljedeći korak je svođenje matrice na Hessenbergovu formu, odnosno na strukturirani oblik koji zadržava iste svojstvene vrijednosti kao početna matrica, ali je znatno pogodniji.³

Hessenbergova forma je poseban strukturirani oblik kvadratne matrice u kojem su svi elementi ispod prve poddijagonale jednaki nuli. Preciznije, matrica $(H=[h_{ij}])$ je **gornja**

³ W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes in FORTRAN 77: The Art of Scientific Computing*, 2nd ed. Cambridge: Cambridge University Press, 1992, str. 476

Hessenbergova ako vrijedi $h_{ij}=0$ za $i>j+1$. Ovaj oblik je posebno pogodan u numeričkim postupcima, jer zadržava iste svojstvene vrijednosti kao početna matrica, a istovremeno omogućava efikasnije izvođenje daljih transformacija.

Redukcija na gornju Hessenbergovu formu zasniva se na nizu ortogonalnih transformacija koje se realizuju Householderovim refleksijama. U k-tom koraku konstruiše se reflektor koji poništava sve elemente u k-toj koloni ispod prve poddijagonale, pri čemu se reflektor primjenjuje s lijeve i s desne strane kako bi transformacija bila sličnosna i zadržala spektar. Nakon (n-2) koraka dobija se matrica (H) koja je gornja Hessenbergova i spektralno ekvivalentna početnoj matrici (A).

$H \leftarrow A$

for k = 0 to n-3:

$x \leftarrow [H(k+1,k), H(k+2,k), \dots, H(n-1,k)]^T$

if $\|x\|_2 = 0$:

nastavi

$\alpha \leftarrow \|x\|_2$

if $x_1 \geq 0$:

$\alpha \leftarrow -\alpha$

$e_1 \leftarrow [1, 0, \dots, 0]^T$

$v \leftarrow x - \alpha e_1$

$\beta \leftarrow 2 / (v^T v)$

for j = k to n-1:

$s \leftarrow v^T \cdot H(k+1:n-1, j)$

```

    H(k+1:n-1, j) ← H(k+1:n-1, j) - β v s

for i = 0 to n-1:

    s ← H(i, k+1:n-1) · v

    H(i, k+1:n-1) ← H(i, k+1:n-1) - β s vT

for i = k+2 to n-1:

    H(i, k) ← 0

end

return H

```

Pseudokod 1. Redukcija matrice na Hessenbergovu formu

Pseudokod je formulisan kao standardna Householder redukcija na gornju Hessenbergovu formu, uz zapis prilagođen implementacijskom obliku (u mjestu) i eksplicitno računanje koeficijenta β .⁴ U svakom koraku spoljne petlje, algoritam izoluje vektor x koji sadrži elemente k -te kolone ispod subdiagonale, te konstruiše Householderov reflektor v sa ciljem poništavanja tih elemenata uz očuvanje euklidske norme vektora. Ključni dio procesa ogleda se u primjeni sličnosti, gdje se unutrašnjim petljama transformacija vrši sukcesivno s lijeve i desne strane (formirajući $P_k^T H P_k$); lijevo množenje vrši eliminaciju elemenata u koloni, dok desno množenje ažurira redove kako bi se očuvale svojstvene vrijednosti matrice bez narušavanja nula kreiranih u prethodnim koracima. Ovakav pristup garantuje numeričku stabilnost kroz pažljiv odabir parametra α , a rezultat je matrica kod koje su svi elementi ispod prve subdiagonale svedeni na nulu, čime se složenost daljih spektralnih proračuna redukuje sa kubne na kvadratnu po iteraciji.

1.5 QR algoritam

⁴ D. Bindel, *Numerical Analysis (CS 4220) – Hessenberg via Householder*, Spring 2016 (teoretska osnova)

Nakon izlaganja teorijskih osnova i pripremni transformacija, konačno dolazimo do centralnog dijela ovog seminarskog rada, a to je **QR algoritam za nalaženje sopstvenih vrijednosti matrica uz prethodno svođenje matrice na Hessenbergovu normalnu formu**. U nastavku će biti detaljno analizirana struktura samog algoritma, način njegove implementacije te uloga Hessenbergove redukcije u postizanju numeričke efikasnosti i stabilnosti postupka.

U okviru ovog rada, fokus je stavljen na osnovnu varijantu QR algoritma nad matricama svedenim na Hessenbergovu formu, kako bi se jasno ilustrovao iterativni proces transformacije i konvergencije ka gornjoj trougaonoj formi. Važno je napomenuti da se u komercijalnim softverskim rješenjima i naprednim numeričkim paketima rijetko koristi ova bazična verzija zbog njene relativno spore, linearne konvergencije.

Umjesto toga, primjenjuju se strategije **jednostrukog i dvostrukog pomaka** (eng. *shifted QR iteration*), koje drastično ubrzavaju proces izolacije sopstvenih vrijednosti. Iako ove tehnike omogućavaju kvadratičnu konvergenciju i efikasno rukovanje kompleksnim parovima korijena, njihova detaljna matematička analiza izlazi iz opsega ovog rada.

Osnovni oblik ovog algoritma počinje svođenjem matrice na Hessenbergovu formu, te je cilj iterativno transformisati matricu tako da ona u svakom koraku sve više liči na gornju trougaonu, jer se u tom graničnom obliku svojstvene vrijednosti nalaze direktno na njenoj dijagonali.

U svakoj iteraciji, algoritam vrši QR dekompoziciju trenutne matrice P , a zatim formira novu matricu kao produkt $R \cdot Q$. Ovakav postupak osigurava da nova matrica P zadrži potpuno iste svojstvene vrijednosti kao i polazna, ali sa smanjenim magnitudama elemenata na subdijagonali. Petlja se ponavlja sve dok se ne zadovolji uslov konvergencije, odnosno dok svi elementi ispod glavne dijagonale ne postanu zanemarivo mali (u pseudokodu predstavljeno kao nula), čime se postiže finalni cilj, što je izolacija spektra matrice kroz numerički stabilan proces koji izbjegava direktno rješavanje karakterističnog polinoma.

kraj ← netačno

$A_{\text{iter}} \leftarrow \text{svediNaHessenberga}(\text{Matrica_S})$

while kraj = netačno do

 kraj $\leftarrow$ tačno

$Q, R \leftarrow \text{qr_faktorizacija}(A_{\text{iter}})$

$A_{\text{iter}} \leftarrow R * Q$

 for i from 2 to n do

 for j from 1 to i-1 do

 if $\text{abs}(A_{\text{iter}}[i, j]) > \text{tolerancija}$ then

 kraj $\leftarrow$ netačno

 end_if

 end_for

 end_for

end_while

// svojstvene vrijednosti se nalaze na glavnoj dijagonali

for i from 1 to n do

$\text{lambda}[i] \leftarrow A_{\text{iter}}[i, i]$

end_for

return lambda

Pseudokod 2. Osnovna verzija QR algoritma.

Ovaj pseudokod strukturiran je tako da kroz iterativni proces transformiše matricu u gornju trougaonu formu, pri čemu se na samom početku vrši poziv funkcije za redukciju na Hessenbergovu formu, što je postupak koji je prethodno detaljno opisan i implementiran. Glavni dio algoritma oslanja se na while petlju koja kontroliše proces konvergencije, unutar nje se u svakom koraku vrši QR faktorizacija trenutne matrice, koristeći neku od klasičnih metoda obrađenih na kursu, nakon čega se faktori Q i R množe obrnutim redoslijedom kako bi se dobila nova, slična matrica. Svrha ugniježđenih for petlji unutar glavnog ciklusa je sistematična provjera elemenata ispod glavne dijagonale, ukoliko su svi ti elementi postali zanemarivo mali (odnosno manji od definisane tolerancije), varijabla *kraj* ostaje u stanju koje prekida dalju iteraciju. Konačno, nakon što algoritam postigne zadovoljavajuću preciznost, posljednja petlja vrši ekstrakciju elemenata sa glavne dijagonale rezultujuće matrice, koji prema Schurovoj formi predstavljaju tražene svojstvene vrijednosti polaznog sistema.

1.6 Vremenska kompleksnost QR algoritma

U analizi ukupne računske kompleksnosti algoritma, neophodno je posmatrati proces kroz dvije glavne faze, a to su inicijalna redukcija i iterativni ciklus pronalaženja svojstvenih vrijednosti. Prva faza, redukcija na Hessenbergovu formu primjenom Householderovih refleksija (Pseudokod 1), zahtijeva približno $10n^3/3^5$ operacija s pomičnim zarezom. Ova faza je ključna jer transformiše punu matricu u strukturu kod koje su svi elementi ispod prve subdijagonale nule, što direktno utiče na efikasnost svake naredne iteracije.

Nakon inicijalne redukcije, započinje QR algoritam u kojem svaka iteracija podrazumijeva dekompoziciju i ponovno množenje faktora. Ukupna složenost algoritma bez primjene pomaka izražava se kao $O(10n^3/3 + kf(n))$, gdje k predstavlja broj iteracija do konvergencije, a $f(n)$ cijenu jedne QR iteracije. Ključna prednost uvođenja

⁵ University of Waterloo, „Lecture 16: Eigenvectors / Eigenvalues - Practical QR,” CS 475 Lecture Notes. [Online]. Dostupno na: <https://student.cs.uwaterloo.ca/~cs475/CS475-Lecture16.pdf>. [Pristupljeno: 21. februar 2026].

Hessenbergove forme ne leži u smanjenju broja iteracija k , već u drastičnom smanjenju kompleksnosti funkcije $f(n)$.

Dok bi za punu matricu jedna QR iteracija (faktorizacija i množenje) zahtijevala $O(n^3)$ operacija, struktura Hessenbergove forme omogućava da se, koristeći optimizovani Gram-Schmidtov proces ili Givensove rotacije, ovaj proces svede na svega $O(n^2)$ operacija po iteraciji. Razlog leži u tome što Hessenbergova matrica već posjeduje nule na svim pozicijama ispod prve subdiagonale, pa algoritam mora „poništit” samo $n-1$ elemenata umjesto njih $n(n-1)/2$.

Poglavlje 2

Implementacija algoritma

U okviru ovog poglavlja predstavljamo implementaciju algoritma za izračunavanje svojstvenih vrijednosti. Proces započinje fazom pripreme, gdje se polazna matrica, primjenom Householderovih refleksija opisanih u **Pseudokodu 1**, redukuje na Hessenbergovu formu kako bi se osigurala računska efikasnost u kasnijim koracima. Centralni dio rješenja oslanja se na iterativni QR algoritam, čiji je osnovni oblik formalizovan kroz pseudokod koji sukcesivno vrši faktorizaciju i rekonstrukciju matrice sve do postizanja Schurove forme. Kompletan postupak je finalizovan razvojem funkcije u programskom jeziku **Julia**, koja koristi ugrađenu qr faktorizaciju za stabilno i precizno određivanje spektra matrice.

```
using LinearAlgebra
```

```
function svediNaHessenberga(A::Matrix{Float64})
```

```
    H = copy(A)
```

```
    n = size(H, 1)
```

```
    for k in 1:n-2
```

```
        x = H[k+1:n, k]
```

```
        norm_x = norm(x)
```

```
        if norm_x == 0 continue
```

```
    end
```

```
    alpha = x[1] >= 0 ? -norm_x : norm_x
```

```
    v = copy(x)
```

```
    v[1] -= alpha
```

```
    beta = 2.0 / dot(v, v)
```

```
    for j in k:n
```

```

        s = beta * dot(v, H[k+1:n, j])
        H[k+1:n, j] -= v * s
    end
    for i in 1:n
        s = beta * dot(H[i, k+1:n], v)
        H[i, k+1:n] -= (s * v)'
    end
    for i in k+2:n
        H[i, k] = 0.0
    end
end
return H
end

```

```

function izracunajSvojstveneVrijednosti(Matrica_S::Matrix{Float64}; tolerancija=1e-12)
    n = size(Matrica_S, 1)
    kraj = false
    A_iter = svediNaHessenberga(Matrica_S)
    while !kraj
        kraj = true
        F = qr(A_iter)
        Q = Matrix(F.Q)
        R = F.R
        A_iter = R * Q
        for i in 2:n
            for j in 1:i-1
                if abs(A_iter[i, j]) > tolerancija
                    kraj = false
                    break
                end
            end
        end
    end
end

```

```

        end
    end
    if !kraj break end
end
end
lambda = [A_iter[i, i] for i in 1:n]
return lambda
end

```

Kod 1. QR algoritam za određivanje svojstvenih vrijednosti sa prethodnom redukcijom na Hessenbergovu formu

Testni primjer 1 - Simetrična matrica.

```

A_test = [ 5.0  2.0  1.0  0.0 ;
           2.0  8.0  3.0  1.0 ;
           1.0  3.0  4.0  2.0 ;
           0.0  1.0  2.0  6.0 ]

println("Testna matrica A_test:")
display(A_test)

referentne_vrijednosti = sort(eigvals(A_test), rev=true)
tvoji_rezultati = izracunajSvojstveneVrijednosti(A_test, tolerancija=1e-12)

println("Referentne vrijednosti: ", referentne_vrijednosti)
println("Rezultati algoritma:    ", moji_rezultati)

razlika = norm(referentne_vrijednosti - moji_rezultati)
println("Numerička razlika: ", razlika)

```

Prvi blok koda kreira matricu 4x4 sa specifičnim vrijednostima. Odabrana je matrica sa realnim elementima kako bi se lakše pratila konvergencija elemenata na dijagonali, te je u ovom testu ta matrica simetrična. Kod prvo poziva ugrađenu Julijinu funkciju. Ovo služi kao kontrolna grupa. Rezultati ove funkcije se sortiraju kako bi se mogli direktno uporediti sa izlazom naše implementacije. Pozivom implementirane funkcije dobijamo naše rezultate, te na kraju računamo normu razlike između dobijenih rezultata i referentnih vrijednosti, što predstavlja finalnu ocjenu tačnosti koda.

Rezultati testnog primjera 1.

Testna matrica A_test:

4×4 Matrix{Float64}:

5.0 2.0 1.0 0.0

2.0 8.0 3.0 1.0

1.0 3.0 4.0 2.0

0.0 1.0 2.0 6.0

Referentne vrijednosti: [11.100779467918178, 6.068841615414279, 3.9013936703764136, 1.928985246291128]

Rezultati algoritma: [11.100779467918182, 6.068841615414284, 3.9013936703764096, 1.9289852462911283]

Numerička razlika: 7.552781223416605e-15

Dobijeni rezultati testiranja jasno potvrđuju izuzetnu preciznost implementiranog QR algoritma, s obzirom na to da se izračunate svojstvene vrijednosti gotovo savršeno poklapaju sa referentnim vrijednostima dobijenim putem standardne Julijine funkcije eigvals. Numerička razlika ukazuje na to da je ukupna greška svedena na nivo mašinske preciznosti, što direktno dokazuje da su teorijske postavke o konvergenciji ka gornjoj trougaonoj formi uspješno prenesene u programski kod.

Posebno je značajno primijetiti da su sve četiri svojstvene vrijednosti realne i stabilne, što potvrđuje da je početna redukcija matrice na Hessenbergovu formu, kao i naknadna QR faktorizacija, izvedena bez gubitka ključnih informacija o spektru matrice. Ovakav

nivo podudarnosti verifikuje ispravnost kriterija zaustavljanja unutar petlje i garantuje pouzdanost algoritma u praktičnim numeričkim primjenama.

Testni primjer 2-Opšta kvadratna matrica(nesimetrična).

```
A_test2 = [ 10.0  1.0  2.0  3.0 ;  
           4.0 12.0  1.0  0.0 ;  
           2.0  1.0  8.0  2.0 ;  
           1.0  0.0  5.0 15.0 ]
```

```
println("Matrica A_test2:")  
display(A_test2)
```

```
referentne_vrijednosti2 = sort(eigvals(A_test2), rev=true)  
moji_rezultati2 = izracunajSvojstveneVrijednosti(A_test2, tolerancija=1e-14)  
println("Referentne vrijednosti: ", referentne_vrijednosti2)  
println("Rezultati algoritma: ", moji_rezultati2)
```

```
razlika2 = norm(referentne_vrijednosti2 - moji_rezultati2)  
println("Numerička razlika : ", razlika2)
```

Test je koncipiran identično kao prethodni, s tim da je uzeta nesimetrična kvadratna matrica.

Rezultati testnog primjera 2.

```
Matrica A_test2:  
4×4 Matrix{Float64}:  
10.0  1.0  2.0  3.0  
 4.0 12.0  1.0  0.0  
 2.0  1.0  8.0  2.0  
 1.0  0.0  5.0 15.0
```

Referentne vrijednosti: [17.478207685532958, 12.59726581200931, 8.31385132702504, 6.610675175432696]

Rezultati algoritma: [17.478207685532922, 12.597265812009308, 8.313851327025041, 6.610675175432679]

Numerička razlika : 3.941151002410555e-14

Drugi testni primjer potvrđuje da algoritam zadržava izuzetnu stabilnost i na asimetričnim matricama, postižući podudarnost sa referentnim vrijednostima na četrnaest decimalnih mjesta. Rezultati demonstriraju da početna redukcija na Hessenbergovu formu uspješno priprema matricu za brzu konvergenciju, bez obzira na nedostatak simetrije u ulaznim podacima.

Testni primjer 3 - Pascalova matrica.

Za ovaj test koristimo Pascalovu matricu (simetričnu pozitivno definitnu matricu), koja je poznata u numeričkoj analizi po tome što njene svojstvene vrijednosti mogu varirati kroz nekoliko redova veličine, što je odličan test za preciznost QR algoritma.

```
A_test3 = [ 1.0  1.0  1.0  1.0 ;  
           1.0  2.0  3.0  4.0 ;  
           1.0  3.0  6.0 10.0 ;  
           1.0  4.0 10.0 20.0 ]
```

```
println("Matrica A_test3:")  
display(A_test3)
```

```
referentne_vrijednosti3 = sort(eigvals(A_test3), rev=true)  
moji_rezultati3 = izracunajSvojstveneVrijednosti(A_test3, tolerancija=1e-14)  
println("Referentne vrijednosti: ", referentne_vrijednosti3)  
println("Rezultati algoritma: ", moi_rezultati3)
```

```
razlika3 = norm(referentne_vrijednosti3 - moji_rezultati3)
println("Numerička razlika: ", razlika3)
```

Rezultati testnog primjera 3.

Matrica A_test3:

4×4 Matrix{Float64}:

1.0 1.0 1.0 1.0

1.0 2.0 3.0 4.0

1.0 3.0 6.0 10.0

1.0 4.0 10.0 20.0

Referentne vrijednosti: [26.304703267097874, 2.2034461676473245, 0.453834550025667,
0.038016015229141054]

Rezultati algoritma: [26.304703267097864, 2.2034461676473236, 0.45383455002566564,
0.0380160152291397]

Numerička razlika: 1.086929612968646e-14

Treći testni primjer na Pascalovoj matrici pokazuje sposobnost algoritma da precizno identifikuje svojstvene vrijednosti čak i kada su one numerički razmaknute, u rasponu od preko 20 do vrijednosti bliskih nuli. Postignuta numerička razlika potvrđuje da redukcija na Hessenbergovu formu i naknadne QR iteracije zadržavaju visok nivo tačnosti bez obzira na varijaciju u veličini svojstvenih vrijednosti.

Testni primjer 4 - Hilbertova matrica.

Kao četvrti i posljednji primjer, možemo koristiti Hilbertovu matricu. Ona je u numeričkoj analizi poznata kao loše uslovljena matrica, što znači da čak i najmanje greške u računanju mogu dovesti do velikih odstupanja u rezultatima.

```
A_test4 = [ 1.0    0.5    0.33333333 0.25 ;
            0.5    0.33333333 0.25    0.2 ;
            0.33333333 0.25    0.2    0.16666667 ;
```

```
0.25    0.2    0.16666667 0.14285714 ]
```

```
println("Matrica A_test4:")
```

```
display(A_test4)
```

```
referentne_vrijednosti4 = sort(eigvals(A_test4), rev=true)
```

```
moji_rezultati4 = izracunajSvojtstveneVrijednosti(A_test4, tolerancija=1e-14)
```

```
println("Referentne vrijednosti: ", referentne_vrijednosti4)
```

```
println("Rezultati algoritma: ", moji_rezultati4)
```

```
razlika4 = norm(referentne_vrijednosti4 - moji_rezultati4)
```

```
println("Numerička razlika: ", razlika4)
```

Rezultati testnog primjera 4.

Matrica A_test4:

4×4 Matrix{Float64}:

```
1.0    0.5    0.333333 0.25
```

```
0.5    0.333333 0.25    0.2
```

```
0.333333 0.25    0.2    0.166667
```

```
0.25    0.2    0.166667 0.142857
```

Referentne vrijednosti: [1.500214278035128, 0.16914122273263205,
0.006738270913703495, 9.669831853543458e-5]

Rezultati algoritma: [1.5002142780351284, 0.16914122273263188, 0.006738270913703514,
9.66983185355211e-5]

Numerička razlika: 4.8249179649021e-16

Uspješno izračunavanje i minimalna numerička razlika u ovom slučaju potvrđuju da Householderova redukcija na Hessenbergovu formu pruža izuzetnu stabilnost čak i kod numerički nestabilnih problema.

Poglavlje 3

Primjena algoritma u praksi

Primjena QR algoritma je fundamentalna u svim oblastima moderne nauke i inženjerstva gdje je neophodno izvršiti spektralnu analizu matrica, odnosno precizno odrediti njihove svojstvene vrijednosti. Budući da svojstvene vrijednosti definišu inherentna svojstva linearnih sistema, ovaj algoritam se koristi za rješavanje kompleksnih problema u stabilnosti konstrukcija, kvantnoj mehanici i analizi dinamičkih sistema. Njegova snaga leži u numeričkoj stabilnosti i sposobnosti da efikasno obradi velike skupove podataka, što ga čini standardnim alatom u bibliotekama za naučno izračunavanje.

U sferi analize podataka i mašinskog učenja, QR algoritam ima direktnu i kritičnu primjenu kroz nekoliko ključnih tehnika, a jedna od njih je analiza glavnih komponenti (PCA)⁶, gdje se QR algoritam koristi kao tehnika za redukciju dimenzionalnosti podataka koja se primarno oslanja na dekompoziciju svojstvenih vrijednosti kako bi se identifikovali najznačajniji pravci varijacije u skupu podataka. Spektralno klasterovanje predstavlja naprednu tehniku grupisanja podataka (klasterizacije) koja koristi svojstvene vrijednosti laplasijan matrice grafa kako bi identifikovala prirodne zajednice ili klastere unutar podataka, te je QR algoritam centralni algoritam koji se koristi za ovo. Još jedna primjena je u latentnoj semantičkoj analizi (LSA). Ova tehnika analize teksta koristi dekompoziciju svojstvenih vrijednosti (kroz SVD proces koji se često oslanja na QR korake) za identifikaciju skrivenih (latentnih) tema i semantičkih veza unutar velikih korpusa dokumenata.

Pored primjene u analizi podataka, QR algoritam ima fundamentalnu ulogu u rješavanju kompleksnih problema unutar fizike i inženjerstva, gdje se dekompozicija svojstvenih vrijednosti koristi za razumijevanje fundamentalnih prirodnih zakona i stabilnosti

⁶ Shlens, "A Tutorial on Principal Component Analysis," Google Research. [Online]. Dostupno na: <https://builtin.com/data-science/step-step-explanation-principal-component-analysis>. [Pristupljeno: 24. feb. 2026].

sistema. U strukturnoj analizi, ovaj algoritam je nezaobilazan alat za proučavanje vibracionih modova različitih konstrukcija, omogućavajući inženjerima da predvide kako će se mostovi ili zgrade ponašati pod uticajem spoljašnjih sila. Slično tome, u teoriji upravljanja, dekompozicija se primjenjuje za detaljnu analizu stabilnosti kontrolnih sistema, što je ključno za automatizaciju i precizno navođenje u modernoj tehnologiji. Na nivou fundamentalne nauke, QR algoritam se koristi u kvantnoj mehanici za rješavanje Schrödingerove jednačine, pretvarajući kompleksne diferencijalne probleme u rješive matrične sisteme čije svojstvene vrijednosti predstavljaju dozvoljene energetske nivoe čestica.

Poglavlje 4

Zaključak i diskusija

Tema numeričkog određivanja svojstvenih vrijednosti matrica ima veliki značaj u numeričkoj linearnoj algebri i brojnim primjenama u inženjerstvu i računarstvu, jer se svojstvene vrijednosti pojavljuju u problemima stabilnosti, vibracija, optimizacije i analize podataka. Zbog numeričke nestabilnosti i nepraktičnosti direktnog pristupa preko karakterističnog polinoma, savremene metode se oslanjaju na stabilne transformacije sličnosti i faktORIZACIJE, a QR algoritam predstavlja jednu od najvažnijih i najstandardnijih tehnika za računanje spektra matrice.

Cilj ovog rada bio je prikazati i implementirati postupak kojim se svojstvene vrijednosti dobijaju primjenom QR iteracije, uz prethodno svođenje matrice na gornju Hessenbergovu formu, te provjeriti tačnost dobijenih rezultata na različitim primjerima. U postavljenom cilju se u velikoj mjeri uspjelo, jer implementirani algoritam daje svojstvene vrijednosti koje se poklapaju s referentnim rezultatima i pokazuje očekivano ponašanje na više tipova matrica, čime je potvrđena korektnost pristupa i njegova praktična upotrebljivost.

Što se tiče performansi, redukcija na Hessenbergovu formu je presudan korak jer značajno smanjuje trošak svake naredne QR iteracije zahvaljujući strukturi matrice u kojoj su elementi ispod prve poddijagonale jednaki nuli. Time iteracije postaju znatno brže nego u slučaju rada nad punom matricom, ali ukupno vrijeme i dalje zavisi od broja iteracija potrebnih za konvergenciju, koji može biti veći kod nesimetričnih i loše uslovljenih matrica. Upravo zbog toga, prirodan prostor za optimizaciju je u ubrzavanju konvergencije i smanjenju broja iteracija, što se u praksi postiže uvođenjem pomaka (shiftova) u QR iteraciju, što je bilo i spomenuto u teoretskom razmatranju, deflacijom već konvergiranih blokova, kao i korištenjem implicitnog QR postupka umjesto eksplicitnog računanja QR faktORIZACIJE u svakoj iteraciji. Dodatno, u slučaju simetričnih matrica, algoritam se može specijalizovati svođenjem na tridiagonalnu formu, čime se dobija još veća efikasnost i pouzdanija konvergencija.

Zaključno, rad potvrđuje da kombinacija Hessenbergove redukcije i QR iteracije daje stabilan i efikasan način za računanje svojstvenih vrijednosti, pri čemu osnovna varijanta algoritma već obezbjeđuje tačne rezultate, dok se predloženim izmjenama može dostići još viši nivo performansi i stabilnosti.

Reference

- [1] Built In, “A Step-by-Step Explanation of Principal Component Analysis (PCA),” *Built In*, [Online]. Available: <https://builtin.com/data-science/step-step-explanation-principal-component-analysis>. Accessed: Feb. 24, 2026.
- [2] W. H. Press *et al.*, “Section 11.5 (PDF),” *Numerical Recipes* (online book PDF), [Online]. Available: <https://phys.uri.edu/nigh/NumRec/bookfpdf/f11-5.pdf>. Accessed: Feb. 24, 2026.
- [3] P. Arbenz, “Lecture Notes, Chapter 4 (PDF),” ETH Zürich, [Online]. Available: <https://people.inf.ethz.ch/arbenz/ewp/Lnotes/chapter4.pdf>. Accessed: Feb. 24, 2026.
- [4] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed. (PDF), [Online]. Available: <https://math.ecnu.edu.cn/~jypan/Teaching/books/2013%20Matrix%20Computations%204th.pdf>. Accessed: Feb. 24, 2026.
- [5] L. N. Trefethen and D. Bau III, *Numerical Linear Algebra* (PDF), [Online]. Available: https://davidtabora.wordpress.com/wp-content/uploads/2015/01/lloyd_n-trefethen_david_bau_iii_numerical_line.pdf. Accessed: Feb. 24, 2026.
- [6] D. Bindel, “Lecture Notes (CS 4220), 2016-03-11 (PDF),” Cornell University, [Online]. Available: <https://www.cs.cornell.edu/~bindel/class/cs4220-s16/lec/2016-03-11-notes.pdf>. Accessed: Feb. 24, 2026.
- [7] Ž. Jurić, *Numerički algoritmi*. Sarajevo: Univerzitet u Sarajevu, Elektrotehnički fakultet, 2018